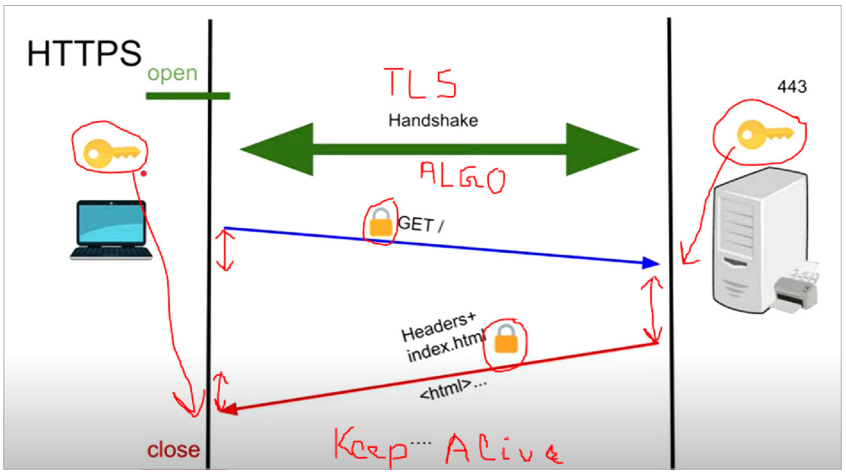

Figure 2: Anatomy of request/response transfer

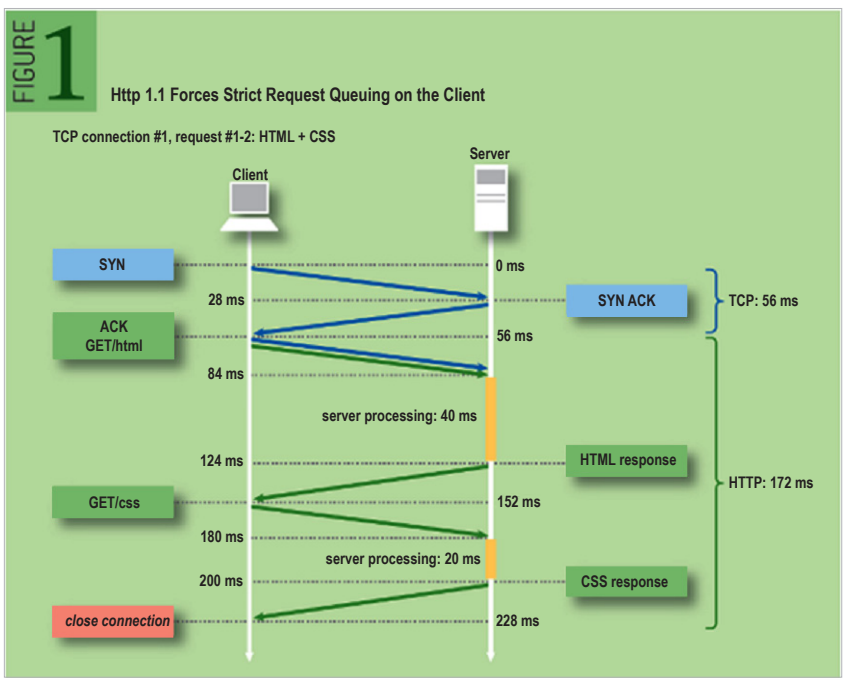

Figure 3: Anatomy of HTTP request [Ref: *https://queue.acm.org/detail.cfm?id=2555617*]

From a Web scalability perspective, the most important layers are:

1. *Application (Layer-7):* Http and WebWorker (WSS)
2. *Session (Layer-5):* SSL/TLS
3. *Transport (Layer-4):* TCP, UDP, ICMP

Let us now go back to when an application sends the data over the OSI stack to see how things work – the anatomy of request/response transfer over a network.

When a request is sent, the TLS handshake is established for setting up a secure channel; this is preferred over plain text communication to avoid any potential security vulnerabilities like Man-in-the-Middle attacks, etc. This is the usual public key infrastructure (PKI) stuff where the public key is passed around and used for encryption purposes. The private key always stays *in-situ* to decrypt the encrypted data at the other end. The time taken to complete the secure channel setup is called *connect* time.

Once the secure channel is set up, the application makes a request. The request takes some time to go to the server. This is called the *send* time.

Once the data is available to the server, it does some processing. This is the processing time component of the HTTP request, called *wait* time. Then the data travels back as a response from the server to the client.

The time spent is called the receive time. Figure 3 depicts all of the above.

Note: *Blocked* returns the time in milliseconds spent waiting for a network connection.

Connect returns the time in milliseconds spent making the TCP connection (0ms to 56ms).

Send returns the time in milliseconds spent sending the request to the server (56ms to 84ms).

Wait returns the time in milliseconds spent waiting for the first byte of response from the server (84ms to 124ms).

Receive returns the time in milliseconds spent reading the complete response from the server (124ms to 152ms).

Now let us look at the legacy version of HTTP, which is http1.1. The way typical modern Web browsers handle http-1.1 is depicted in Figure 4.

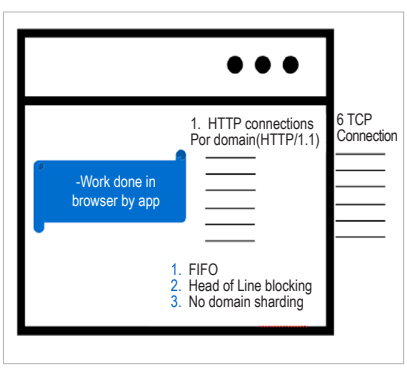

Figure 4: Browser and http1.1